

LAB 3.2: Xử lý mất cân bằng dữ liệu

I. Giới thiệu

Dữ liệu mất cân bằng, giống như tên gọi, xảy ra khi dữ liệu có sự phân bố không đồng đều giữa các lớp (một lớp trong tập dữ liệu có số lượng mẫu áp đảo so với các lớp còn lại). Trong các bài toán phân loại, điều này có thể khiến cho mô hình bị thiên lệch, tập trung quá mức vào lớp chiếm đa số và bỏ qua các đặc trưng quan trọng của lớp thiểu số. Hậu quả là mô hình kém hiệu quả trong việc nhận diện các trường hợp hiếm gặp, làm giảm độ chính xác tổng thể.

Tình trạng này thường xuất hiện trong thực tế, chẳng hạn như trong y tế, số ca mắc bệnh thường ít hơn đáng kể so với số ca không mắc, hoặc trong lĩnh vực tài chính, các giao dịch gian lận chiếm tỷ lệ rất nhỏ so với giao dịch hợp lệ. Nếu không xử lý đúng cách, mô hình có thể đơn giản phớt lờ lớp thiểu số và đưa ra dự đoán sai lệch.

Để khắc phục vấn đề, ta cần áp dụng các kỹ thuật xử lý mất cân bằng dữ liệu nhằm giúp mô hình học được cả hai lớp một cách công bằng hơn. Các kỹ thuật này được chia thành hai nhóm chính.

1.1. Data-level techniques

Là các phương pháp tập trung vào điều chỉnh tập dữ liệu trước khi đưa vào mô hình. Nhóm này bao gồm *Resampling techniques* - thay đổi số lượng mẫu trong các lớp để cân bằng dữ liệu - và *Data augmentation* - tạo ra các mẫu mới từ các mẫu hiện có bằng cách áp dụng các phép biến đổi, thay đổi một số đặc trưng hoặc thêm nhiễu. Kỹ thuật này thường được sử dụng trong các bài toán phân loại ảnh, nhưng cũng có thể áp dụng cho dữ liệu dạng bảng.

1.1.1. Oversampling

a) Random Oversampling

Tăng số lượng mẫu của lớp thiểu số bằng cách sao chép ngẫu nhiên các mẫu hiện có.

b) SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) là một kỹ thuật tạo ra các điểm dữ liệu mới bằng cách nội suy giữa các mẫu dữ liệu hiện có, theo các bước sau:

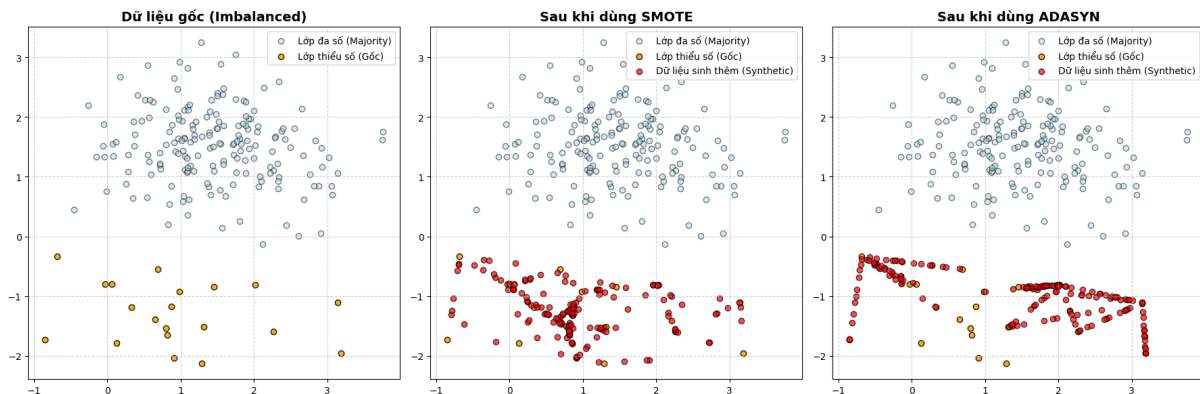
1. Chọn số lượng mẫu cần tạo cho lớp thiểu số (ví dụ: từ 100 lên 200).
2. Với mỗi mẫu trong lớp thiểu số, sử dụng KNN để tìm ra k láng giềng gần nhất.
3. Chọn ngẫu nhiên một trong k láng giềng, tạo ra mẫu mới bằng cách nội suy giữa mẫu gốc và láng giềng được chọn theo công thức:

$$x_{new} = x_i + \lambda(x_{neighbor} - x_i) \quad (1)$$

với λ là một số ngẫu nhiên trong khoảng $[0, 1]$, quyết định vị trí của mẫu mới trên đoạn thẳng nối giữa mẫu gốc và láng giềng.

4. Lặp lại cho đến khi đạt được số lượng mẫu mong muốn trong lớp thiểu số.

c) *ADASYN* phương pháp tương tự như SMOTE nhưng tập trung vào tạo số lượng mẫu khác nhau dựa trên ước lượng phân phối cục bộ của lớp cần được oversample.



1.1.2. Undersampling

a) *Random Undersampling*

Giảm số lượng mẫu của lớp đa số bằng cách loại bỏ ngẫu nhiên các mẫu hiện có.

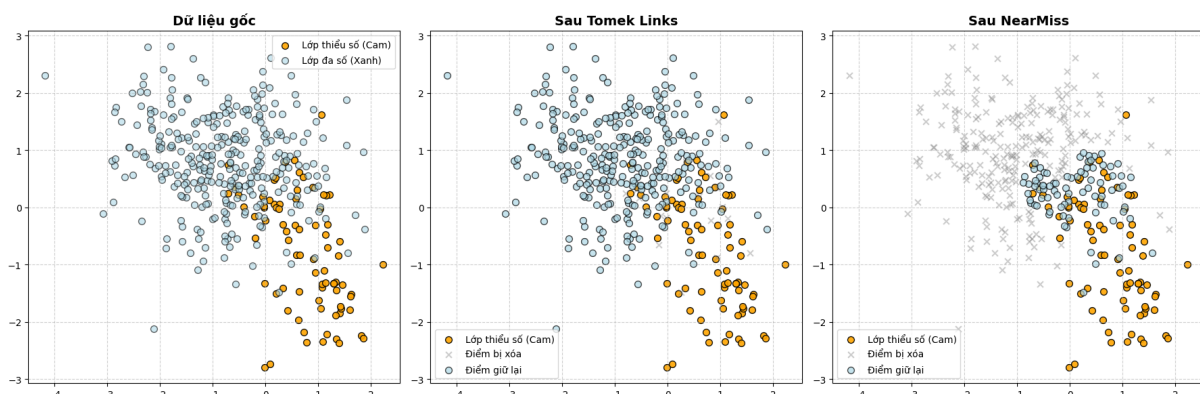
b) *NearMiss*

Là một nhóm các phương pháp Undersampling có chọn lọc, thay vì loại bỏ ngẫu nhiên. Ý tưởng chính là giữ lại các mẫu của lớp đa số **gần** với các mẫu của lớp thiểu số nhất - những mẫu đóng vai trò quan trọng trong việc xác định ranh giới phân loại - đồng thời loại bỏ các mẫu ở lớp đa số nằm xa lớp thiểu số. Quá trình thực hiện gồm ba bước:

1. Xác định số mẫu M cần giữ lại ở lớp đa số;
2. Với mỗi điểm trong lớp đa số, tính khoảng cách tới k láng giềng gần nhất/xa nhất trong lớp thiểu số rồi lấy trung bình;
3. Sắp xếp các mẫu theo khoảng cách trung bình tăng dần và lựa chọn M mẫu có giá trị nhỏ nhất. Việc lựa chọn láng giềng tại lớp thiểu số trong bước (2) tùy thuộc vào tính chất của từng bài toán cụ thể.

c) *Tomek Links*

Loại bỏ các mẫu của lớp đa số gần với lớp thiểu số. Một cặp mẫu (x_i, x_j) được gọi là Tomek Link nếu x_i thuộc lớp thiểu số, x_j thuộc lớp đa số, và hai điểm này là láng giềng gần nhất của nhau theo khoảng cách Euclidean (không có mẫu nào khác nằm giữa chúng). Thuật toán lặp lại việc xác định các Tomek Link và loại bỏ x_j cho đến khi không còn Tomek Link nào.



1.1.3. Hybrid Methods

SMOTE-Tomek kết hợp giữa Oversampling và Undersampling. Đầu tiên, SMOTE được dùng để sinh thêm dữ liệu cho lớp thiểu số; tuy nhiên, quá trình nội suy có thể tạo ra các điểm nhiễu lẫn sâu vào vùng của lớp đa số. Tiếp theo, Tomek Links được áp dụng để dọn dẹp các cặp điểm nằm quá sát nhau trên ranh giới, giúp làm sạch biên phân lớp và tránh overfitting vào các điểm nhiễu.

1.2. Algorithm-level techniques

Là các phương pháp giúp mô hình học tốt hơn từ dữ liệu mất cân bằng, bao gồm:

Class weights: Điều chỉnh trọng số trong hàm mất mát để tăng tầm quan trọng của lớp thiểu số. Ví dụ, trong bài toán Logistic Regression, hàm mất mát (Cross-Entropy) được điều chỉnh thêm các trọng số w_0, w_1 để phạt nặng hơn khi mô hình dự đoán sai lớp thiểu số:

$$Loss = -\frac{1}{N} \sum_{i=1}^N [w_1 y_i \log(\hat{y}_i) + w_0 (1 - y_i) \log(1 - \hat{y}_i)] \quad (2)$$

(Chỉ có thể áp dụng cho một số thuật toán như Logistic Regression, SVM, Decision Trees)

Ensemble methods: Sử dụng các phương pháp như bagging hoặc boosting để cải thiện khả năng phân loại lớp thiểu số.

Cost-sensitive learning: Điều chỉnh chi phí lỗi trong quá trình học bằng cách thiết kế lại hàm mất mát để tính đến chi phí lỗi.

Các phương pháp này có thể được áp dụng riêng lẻ hoặc kết hợp với nhau để tối ưu hóa hiệu suất của mô hình, tùy thuộc vào đặc điểm của tập dữ liệu và bài toán cụ thể. Việc lựa chọn chiến lược phù hợp đóng vai trò quan trọng trong việc cải thiện độ chính xác của mô hình trên lớp thiểu số, giúp tránh tình trạng thiên lệch về lớp đa số.

1.3. Đánh giá hiệu suất mô hình

Khi đánh giá hiệu suất mô hình trên tập dữ liệu mất cân bằng, ta không nên chỉ dựa vào độ chính xác (accuracy), vì nó có thể gây hiểu lầm khi lớp đa số chiếm ưu thế. Ví dụ, nếu một mô hình dự đoán tất cả các mẫu là lớp đa số, độ chính xác có thể đạt 95% trong khi không phát hiện được bất kỳ mẫu nào của lớp thiểu số. Thay vào đó, các chỉ số sau đây sẽ cung cấp cái nhìn toàn diện hơn.

Precision: Cho biết trong số các mẫu được mô hình dự đoán thuộc lớp thiểu số, có bao nhiêu mẫu thực sự đúng.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3)$$

Recall (Sensitivity): Đánh giá khả năng mô hình phát hiện chính xác các mẫu thuộc lớp thiểu số trên tổng số mẫu thực tế của lớp này.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4)$$

F1-score: Trung bình điều hòa của Precision và Recall, giúp cân bằng giữa hai chỉ số này.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5)$$

AUC-ROC: Đánh giá hiệu suất mô hình trên mọi ngưỡng phân loại, thể hiện khả năng phân biệt giữa hai lớp một cách tổng quát.

II. Bài tập thực hành

Các phương pháp upsampling và downsampling nêu trên đã có sẵn tại thư viện `imbalanced-learn`. Trong bài thực hành này, các bạn không cần cài đặt lại.

Bài 1: Xử lý dữ liệu mất cân bằng trong bài toán dự đoán bệnh tiểu đường

Trong bài thực hành tuần trước, hồi quy logistic đã được áp dụng để tiến hành dự đoán nguy cơ mắc bệnh tiểu đường trên tập dữ liệu Pima Indian Diabetes. Tuy nhiên, mô hình hồi quy logistic các bạn huấn luyện đã không thực sự có được hiệu suất tốt trên lớp thiểu số, điều này có thể do dữ liệu mất cân bằng giữa hai lớp. Hãy thực hiện khảo sát lại dữ liệu và áp dụng các kỹ thuật đã biết trong bài thực hành tuần này để cố gắng cải thiện hiệu suất mô hình hồi quy logistic của bạn.

Yêu cầu 1: Khảo sát dữ liệu

- Tiến hành khảo sát lại dữ liệu, xem xét sự phân bố của các lớp trong biến mục tiêu.
- Vẽ biểu đồ phân bố của biến mục tiêu để thấy rõ sự mất cân bằng giữa các lớp.

Yêu cầu 2: Áp dụng các kỹ thuật xử lý mất cân bằng

- Thực hiện tối thiểu 2 trong 4 phương pháp đã được đề cập để áp dụng trên tập huấn luyện.
- Huấn luyện lại mô hình trên từng tập dữ liệu đã xử lý và so sánh kết quả.

Yêu cầu 3: So sánh và phân tích

- So sánh các chỉ số đánh giá (Accuracy, Precision, Recall, F1-score) giữa mô hình gốc và các mô hình sau khi xử lý.
- Vẽ confusion matrix cho từng trường hợp và nhận xét về sự cải thiện trên lớp thiểu số.

Bài 2: Phát hiện gian lận thẻ tín dụng

Tập dữ liệu Credit Card Fraud Detection chứa các giao dịch thẻ tín dụng của người dùng châu Âu. Trong tổng số hàng trăm nghìn giao dịch, chỉ có một lượng rất nhỏ là gian lận. Vì lý do bảo mật, các đặc trưng ban đầu đã được biến đổi qua PCA thành các biến số thực (từ $V1$ đến $V28$), chỉ có biến `Time` và `Amount` là giữ nguyên. Mục tiêu của bài toán là xây dựng mô hình phát hiện các giao dịch gian lận (Class 1) giữa số lượng lớn các giao dịch hợp lệ (Class 0).

Yêu cầu 1: Khảo sát sự mất cân bằng

- Tải tập dữ liệu và tách biến mục tiêu (`Class`).
- Tính toán và in ra tỷ lệ giữa số lượng giao dịch gian lận và giao dịch hợp lệ.
- Vẽ biểu đồ cột (Bar plot) thể hiện số lượng mẫu của hai lớp. Nhận xét về hình dáng của biểu đồ này.

Yêu cầu 2: Huấn luyện mô hình

- Chia dữ liệu thành tập huấn luyện và tập kiểm thử (tỷ lệ 80-20).
*Lưu ý: Sử dụng tham số **stratify** khi chia dữ liệu để đảm bảo tỷ lệ lớp thiểu số được giữ nguyên ở cả hai tập.*
- Huấn luyện mô hình Logistic Regression mặc định (chưa xử lý mất cân bằng) trên tập huấn luyện.

- Dự đoán trên tập kiểm thử, in ra các chỉ số: Accuracy, Precision, Recall, F1-score và vẽ Confusion Matrix.
- **Câu hỏi:** Tại sao Accuracy lại rất cao (có thể đạt $> 99\%$) nhưng mô hình lại được coi là thất bại trong thực tế bài toán này? Chỉ số nào (Precision hay Recall) quan trọng hơn khi ngân hàng muốn bắt gian lận thẻ?

Yêu cầu 3: Áp dụng các kỹ thuật xử lý mất cân bằng

- **Phương pháp Data-level:** Áp dụng ít nhất một phương pháp (ví dụ: SMOTE hoặc NearMiss) trên tập huấn luyện để sinh hoặc giảm mẫu.
- **Phương pháp Algorithm-level:** Khởi tạo một mô hình Logistic Regression mới với tham số `class_weight='balanced'` để tự động điều chỉnh trọng số phạt.
- Huấn luyện lại các mô hình trên tập dữ liệu tương ứng và tiến hành dự đoán trên tập kiểm thử.

Yêu cầu 4: Đánh giá tổng hợp và so sánh

- So sánh các chỉ số (lưu ý F1-score và Recall của lớp gian lận) giữa: mô hình gốc, mô hình dùng phương pháp Data-level, và mô hình dùng phương pháp Algorithm-level.
- Vẽ Confusion Matrix cho từng trường hợp và phân tích sự thay đổi của các trường hợp bị bỏ lọt (False Negative) sau khi áp dụng các kỹ thuật xử lý.